

Signal AI — Intelligence Graph Pipeline

Technical Architecture Document · GraphOne / FrontierAtlas

Pipeline Results Summary

Entity	Count	Primary Source	Status
Research Papers	1,025	Arxiv API + Papers with Code	✓ Done
Startups	1,002	HuggingFace + GitHub + YC	✓ Done
Products	2,081	HuggingFace Models + Spaces	✓ Done
News Articles	19	RSS Feeds + HackerNews (24hr)	✓ Fresh
Job Postings	22	Remote + Arbeitnow (24hr)	✓ Fresh
Entity Mappings	3,105	RapidFuzz Resolution Engine	✓ Done

1. Scale Strategy — Collecting 500,000+ Records

Current Architecture (Demo)

The pipeline uses a single-node async architecture with **asyncio** + **aiohttp**, processing 10 concurrent connections per source. This yields ~5,000 records/hour. All scrapers are stateless and source-agnostic, making horizontal scaling trivial.

Production Architecture for 500k+ Records

Layer	Technology	Role
URL Frontier	Redis SET	Deduplicated crawl queue, SHA256 URL hashing
Task Queue	Celery + Redis	Distributed job dispatch to N worker pods
Scraper Workers	K8s Pods (N=50)	Horizontal scale, each handles 200 req/min
LLM Extraction	Async Groq API	Rate-limited, fallback chain per worker
Entity Resolution	Batch RapidFuzz	Nightly dedup runs on accumulated data
Storage	PostgreSQL + Qdrant	Structured + vector data with full-text search

Throughput Calculation:

- 50 worker pods × 200 requests/min = **10,000 requests/min**
- Average 3 records per page = **30,000 records/min**
- 500,000 records ÷ 30,000/min = **~17 minutes** at full scale
- No code changes required — only add worker pods via K8s HPA

2. Handling 413 (Payload Too Large) & 429 (Rate Limit)

LLM Fallback Chain

Every LLM call passes through a 3-tier fallback chain. Each model is attempted independently, ensuring zero data loss even if primary models are unavailable:

429 Rate Limit Handling

- **Exponential backoff:** wait = 2ⁿ + random(0,1) seconds where n = retry count
- **Max 3 retries** per model before escalating to next in chain
- **Jitter:** random(0,1) added to prevent thundering herd problem
- **Per-model tracking:** each model has independent retry counter
- **Result:** 100% success rate across 30 LLM calls in demo (with auto-recovery)

413 Context Window Prevention

- **Max chunk size:** 3,000 characters (well below 8k token limits)
- **Word-boundary splits:** rfind(' ') prevents mid-word truncation
- **Semantic priority:** title + abstract in first chunk (most information-dense)
- **Multi-chunk strategy:** long documents split into sequential API calls
- **Response assembly:** JSON responses merged post-extraction

3. Freshness Tracking — Never Process Duplicates

Current Implementation

- **24-hour filter:** all news/jobs checked against UTC cutoff timestamp
- **Multi-format date parsing:** ISO-8601, RFC-2822, relative ('2 hours ago')
- **Relative date resolution:** regex patterns convert to absolute UTC timestamps
- **Missing date heuristic:** content hash compared against previous run

Production Freshness Architecture

Component	Implementation	Purpose
URL Dedup	Redis SET + SHA256(url)	Never re-crawl same URL across nodes
Content Dedup	Qdrant vector similarity	Detect near-duplicate articles (>0.95 sim)
Timestamp DB	PostgreSQL last_seen col	Track per-source last successful crawl
TTL Re-crawl	Redis key expiry (24hr)	Auto-schedule re-crawl after TTL expires
Bloom Filter	Redis BF module	O(1) membership check at 10M+ URL scale

4. Storage Strategy

Primary Database: PostgreSQL

Justification: Structured entity data (startups, products, papers) has well-defined schemas with relational links (startup → products, paper → github repo). PostgreSQL provides ACID guarantees, JSON column support for flexible metadata, and pg_trgm for fuzzy text search. Partitioning by recordType enables efficient bulk queries per entity type.

Vector Store: Qdrant

Justification: Entity resolution at 500k+ scale requires semantic similarity beyond fuzzy string matching. Qdrant stores embeddings of entity names + descriptions, enabling cosine similarity search to detect 'OpenAI' vs 'OpenAI Inc.' vs 'Open AI' automatically. Also powers near-duplicate news detection across distributed crawler nodes.

Cache & Queue: Redis

Justification: Redis SET provides O(1) URL deduplication across distributed workers. Redis Streams power the job queue (replacing Celery for simpler deployments). Redis BF (Bloom Filter) module handles 10M+ URL membership checks with minimal memory.

Store	Technology	Data	Scale Target
Primary DB	PostgreSQL 16	Entities, schemas, metadata	500k+ records
Vector Store	Qdrant	Entity embeddings, dedup	10M+ vectors
Cache/Queue	Redis 7	URL frontier, job queue	100M+ keys
Object Store	S3 / MinIO	Raw HTML, PDFs, backups	Unlimited

5. Anti-Bot & Phase V Strategy

Current Implementation

- **Async-first:** asyncio + aiohttp with TCP connection pooling (limit=5 per source)
- **API preference:** Arxiv, HuggingFace, GitHub APIs used instead of HTML scraping
- **User-Agent rotation:** fake-useragent library cycles through real browser strings
- **Polite delays:** asyncio.sleep(0.3–3s) between requests per source

Cloudflare / DataDome Bypass (Production)

- **Playwright Async:** headless Chromium with stealth plugin masks automation signals
- **Residential proxies:** BrightData or Oxylabs rotate IPs per request
- **Human simulation:** random.uniform(2,5)s delays, mouse movement, scroll events
- **Cookie persistence:** session cookies maintained across requests per domain
- **CAPTCHA fallback:** 2captcha API integration for hard-blocked pages
- **TLS fingerprinting:** curl-cffi library mimics real Chrome TLS handshake

6. Tech Stack Summary

Component	Technology	Component	Technology
Runtime	Python 3.11, asyncio	LLM Provider	Groq API
HTTP Client	aiohttp + BeautifulSoup4	Entity Resolution	RapidFuzz
Task Queue	Celery + Redis	Vector Store	Qdrant
Primary DB	PostgreSQL 16	Anti-Bot	Playwright + stealth
Object Store	S3 / MinIO	Monitoring	Prometheus + Grafana
Container	Docker + K8s	CI/CD	GitHub Actions